CAPÍTULO 1

COMPUTADORA DIGITAL

Una computadora digital es una combinación de dispositivos y circuitos electrónicos organizados de tal forma, que pueden realizar una **secuencia programada** de operaciones con un mínimo de intervención humana. A la secuencia de operaciones se le denomina **Programa**. Un programa es un conjunto de instrucciones codificadas que se almacenan en la memoria interna de la computadora junto con todos los datos que el programa requiere. Por otra parte, para que la computadora pueda ser útil, es necesario que ésta interactúe con el exterior pidiendo datos para ser procesados, y muestre de alguna forma los resultados que obtiene.

Modelo De Von Neumann

La mayoría de las computadoras digitales basan su arquitectura en el **modelo de Von Neumann**. En este modelo, una computadora está formada por cuatro unidades que interactúan entre sí. Estas cuatro unidades son: El procesador, La Unidad de Memoria, la Unidad de Entrada-Salida y La Unidad de Control, ver figura 1-1.

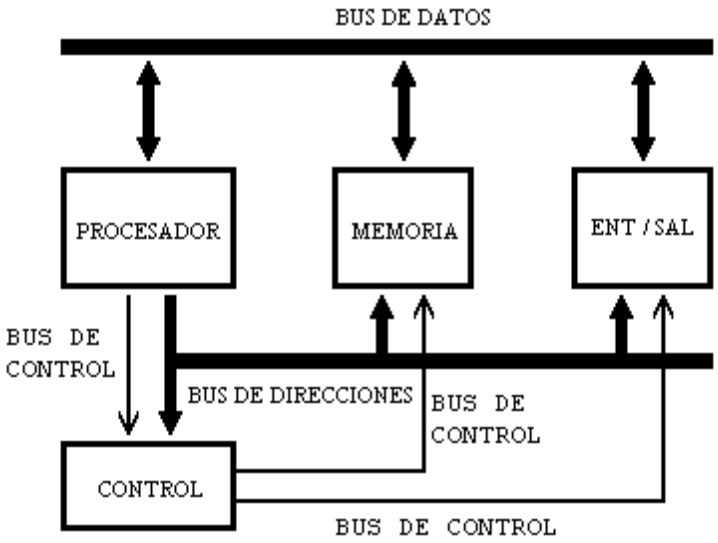


Figura 1-1. Modelo de Von Neumann

La interacción de estas cuatro unidades se lleva a cabo a través de tres conjuntos de conectores llamados conductos o buses: El bus de datos, el bus de direcciones y el bus de control.

El procesador

El procesador es el dispositivo que ejecuta las instrucciones almacenadas en la memoria, el programa. Para lograr esto debe ser capaz de leer y escribir en la memoria y en la unidad de entrada-salida.

Unidad de memoria

Es un conjunto o arreglo de registros binarios de longitud definida (8, 16, 32 o 64 bits), en los cuales se puede almacenar instrucciones codificadas y datos. Cada uno de estos registros se caracteriza por tener una dirección única, por lo que generalmente cada una de estas localidades se identifica por su dirección la cual es la posición que le toca a ese registro dentro del arreglo. Las direcciones se expresan siempre en hexadecimal.

Unidad de entrada-salida

Una unidad de entrada-salida, llamada también puerto, es un dispositivo empleado por la computadora para recibir o enviar datos al mundo exterior. Algunos de estos dispositivos son: Teclado, ratón, interruptores, impresora, monitor, discos magnéticos, tarjetas de comunicación, etc.

Estos dispositivos al igual que las localidades de memoria tienen direcciones únicas por medio de las cuales el procesador interactúa con ellos. Estas direcciones se expresan generalmente en hexadecimal.

Unidad de control

La función de la Unidad de control es coordinar la operación de todas las otras unidades ofreciendo señales de temporización para mantener el sistema en sincronización adecuada y de control para seleccionar si se desea leer o escribir a una localidad de memoria o a un dispositivo de entrada-salida.

Buses de datos, direcciones y control

El procesador utiliza el **bus de datos** para mover datos entre el procesador y la memoria, entre el procesador y los dispositivos de entrada-salida (puertos) o entre la memoria y los dispositivos de entrada-salida. El procesador utiliza el **bus de direcciones**, para indicar la dirección de memoria o puerto sobre la que quiere leer o escribir. Por último el **bus de control** se utiliza para habilitar el dispositivo sobre el que se va a leer o escribir e indicar cual de estas operaciones se va a realizar.

El Procesador

En la figura 1-2 se muestra un modelo simplificado de un procesador o **CPU** (del inglés Central Processing Unit).

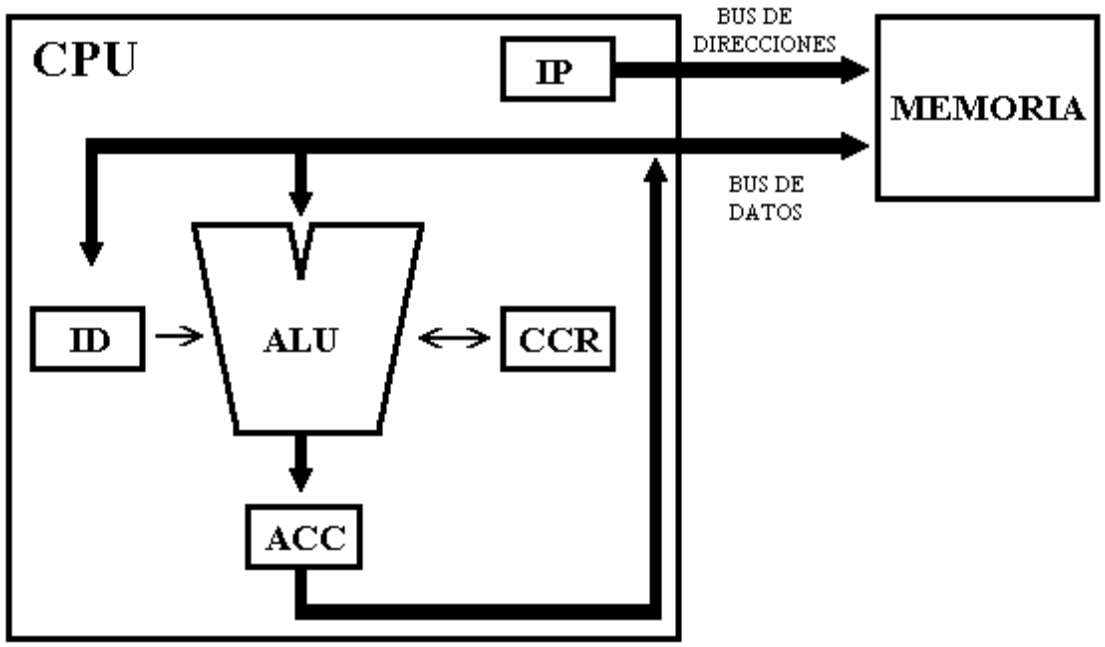


Figura 1-2. Modelo simplificado de un procesador

Generalmente las instrucciones que forman un programa se encuentran almacenadas en forma consecutiva a lo largo de la memoria. Para ejecutar un programa el procesador sólo necesita realizar cíclicamente los siguientes pasos:

1. Lee de la dirección de memoria indicada por el apuntador de instrucciones, IP, el byte de instrucción y la envía al decodificador de instrucciones, ID.

   El **apuntador de instrucciones**, es un registro interno del procesador, el cual contiene la dirección de memoria donde se encuentra la próxima instrucción a ser ejecutada. Al inicio, el apuntador de instrucciones contiene la dirección de la primera instrucción del programa.

2. Actualiza el IP para que apunte al siguiente byte.

3. Identifica o **decodifica** la instrucción mediante la circuitería del ID.
4. Si se requiere, lee el valor del operando de la dirección de memoria indicada por el IP.

5. Si se requiere, Actualiza el IP para que apunte al siguiente byte.

6. Ejecuta la instrucción. El ID genera las señales necesaria para que:

- La unidad aritmética y lógica, ALU, ejecute las operaciones indicadas por la instrucción

- Se almacene el resultado en el acumulador, ACC. El acumulador es un registro utilizado para almacenar el dato sobre los que se va a hacer una operación y el resultado de la operación.

- Se almacene en el registro de código de condiciones, CCR, el estado de la operación realizada. Uno de los bits de ese registro indica si el resultado de la operación fue cero, otro si fue negativo, otro si generó un acarreo, etc. Algunas instrucciones utilizan uno o más de los valores que una instrucción anterior dejó en este registro como parte de sus operandos.

Instrucciones Del Procesador

Cada procesador tiene un conjunto de instrucciones que puede realizar. Por medio de éstas se especifica el programa que queremos que ejecute el procesador. Este conjunto de instrucciones se le conoce como **juego de instrucciones**.

Las mayoría de las instrucciones están formadas por dos partes: la **operación** y los **operandos**. La operación es la acción que debe realizarse. Los operandos son los elementos sobre los que se realizará ésta acción. Por lo tanto, dependiendo de su tipo, las instrucciones pueden tener cero uno o más operandos. Sin embargo en todas las instrucciones existe la operación.

Cada operación está codificada mediante una secuencia binaria única llamada **código de la operación**. Como los operandos son también secuencias binarias, cada instrucción se codifica mediante una secuencia de bytes: uno o más para el código de la operación y cero, uno o más para los operandos. Un programa donde sus instrucciones están codificadas como secuencias binarias está en **lenguaje máquina**.

Para ilustrar lo anterior, supongamos un microprocesador básico en el que los códigos de operación sean de un byte y los operandos sean también de un byte. Supongamos también que el juego de instrucciones que tiene nuestro procesador básico contiene entre otras, las instrucciones mostradas en la tabla 1.1:

Tabla 1.1 Conjunto de instrucciones de un procesador básico.

Instrucción	Descripción	Código De la Operación	Instrucción Codificada
ACC ← *n*	Cargar en el acumulador del procesador un número *n*.	01	01 n
ACC ← [*n*]	Cargar en el acumulador del procesador el contenido de la dirección *n*.	02	02 n
[*n*] ← ACC	Almacenar el contenido del acumulador en la dirección *n*.	03	03 n
ACC ← ACC + *n*	Sumarle al acumulador el número *n* y guardar el resultado en el acumulador.	04	04 n
ACC ← ACC + [*n*]	Sumarle al acumulador el contenido de la dirección *n* y guardar el resultado en el acumulador.	05	05 n

Lenguajes Máquina, Ensamblador Y De Alto Nivel

A grandes rasgos, la evolución de la programación desde la programación en el lenguaje máquina, lenguaje de bajo nivel o ensamblador y los lenguajes de alto nivel es la siguiente:

- Las primeras computadoras eran programadas alimentando manualmente las secuencias binarias que constituían las instrucciones del programa mediante una batería de interruptores. Obviamente programación de esas primeras computadoras era una tarea muy tediosa y sujeta a errores. Posteriormente se substituyó la batería de interruptores por teclados hexadecimales que permitían que el programador cargara una secuencia de cuatro bits presionando una sola tecla del teclado hexadecimal.

- El primer avance significativo en la programación fue la invención del **lenguaje ensamblador**. En el lenguaje ensamblador las operaciones del procesador son codificadas mediante secuencias de caracteres en lugar de secuencias binarias. Estas palabras llamadas **mnemónicos**, son por lo general el nombre de la operación o una abreviatura de éste. En la tabla 1.2 se ilustran algunos mnemónicos de algunas de las instrucciones.

  Adicionalmente, en el lenguaje ensamblador, a las localidades de memoria en las que se almacenan datos se les pueden asignarse nombres simbólicos de modo que esas localidades pueden referenciarse usando sus nombres en lugar de sus direcciones.

Tabla 1.2 Mnemónicos de las instrucciones

Instrucción	Mnemónico	Instrucción en ensamblador
ACC ← *n*	mov	mov ax, n
ACC ← [*n*]	mov	mov ax, [dato]
[*n*] ← ACC	mov	mov [resultado], ax
ACC ← A + *n*	add	add ax, n
ACC ← A + [*n*]	add	add ax, [dato]

Un programa escrito en lenguaje ensamblador es traducido al lenguaje máquina de la computadora donde se va a ejecutar mediante un programa traductor llamado **ensamblador**. Como las instrucciones del lenguaje ensamblador son las mismas que el procesador puede ejecutar, cada procesador tiene su lenguaje ensamblador el cual es diferente del ensamblador de otro procesador.

- El siguiente avance en la programación consistió en la invención de los lenguajes de alto nivel. La principal ventaja de los lenguajes de alto nivel es que las instrucciones en esos lenguajes, se escriben en una forma más entendible por las personas ya que se parecen al lenguaje que empleamos en nuestra vida diaria. Aquí, el programa que traduce el programa escrito en lenguaje de alto nivel a un programa en lenguaje máquina se conoce como **compilador**. Otra ventaja de los lenguajes de alto nivel es que los programas escritos en éstos son hasta cierto punto independientes del procesador en que se van a ejecutar los programas. Para transportar un programa de un procesador a otro, casi siempre es suficiente con compilarlo con un compilador que genere código máquina para el nuevo procesador.

  Las desventajas de los lenguajes de alto nivel con respecto al lenguaje ensamblador son: En primer lugar, la mayoría de los lenguajes de alto nivel limitan al programador en cuanto a las tareas que éste puede hacer y en segundo lugar un programa producido usando un lenguaje de alto nivel es de mayor tamaño y más lento de uno equivalente escrito en lenguaje ensamblador.

Los siguientes dos ejemplos ilustran cómo ocurre la conversión entre las instrucciones de un programa de alto nivel y sus correspondientes instrucciones en lenguaje máquina:

1. Considere la siguiente proposición en el lenguaje C:

   ```
   c = a + b;
   ```

   Si suponemos que las variables a, b, c están en las localidades cuyas direcciones son 25, 33 y 37, respectivamente, entonces la proposición anterior correspondería al siguiente conjunto de instrucciones de nuestro procesador básico:

Instrucciones	Codificación
ACC ← [25]	02
	25
ACC ← ACC + [33]	05
	33
[37] ← ACC	03
	37

2. Considere la siguiente proposición en el lenguaje C:

   z = x + 13 + y

   Si suponemos que las variables x, y, z están en las localidades cuyas direcciones son 23, 44 y 17, respectivamente, entonces la proposición anterior correspondería al siguiente conjunto de instrucciones de nuestro procesador básico:

Instrucciones	Codificación
ACC ← [23]	02
	23
ACC ← ACC + 13	04
	13
ACC ← ACC + [44]	05
	44
[17] ← ACC	03
	17

Problemas

Para cada una de las siguientes proposiciones en el lenguaje C, escriba el correspondiente conjunto de instrucciones que deberá ejecutar el procesador básico y codifíquelas a lenguaje máquina. Añada las instrucciones que sean necesarias.

1. z = x + 13 * y
2. z = (x + 13) * y

Bibliografía

1. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

2. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documentos PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

3. Mano, M. Morris. Lógica Digital y Diseño de Computadores. Prentice-Hall Hispanoamericana, S. A. México. 1982.

4. Mano, M. Morris. Ingeniería Computacional: Diseño del Hardware. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Morgan, Christhopher L, y Waite Mitchel. Introducción al microprocesador 8086/8088 (16 bits). McGraw-Hill/Interamericana de México, S.A de C. V. México. 1988.